

なぜ Linear が既定なのか — 速さではなかった。差は 4%

2026-09-14 | Houdini 21.0.700 | /obj/rigskin | ノード 6 / 接続 7

何を作ったか

実験053で、Dual Quaternion のほうが 154倍も痩せないと分かった。ではなぜ Linear が既定のままなのか。考えられるのは2つ。遅いのか、ねじれで挙動が違うのか。

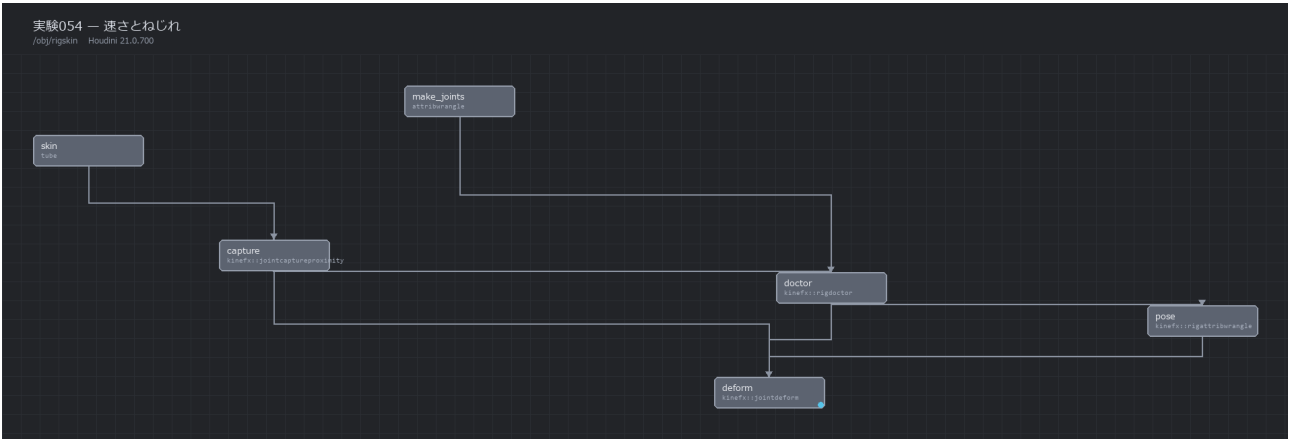
速さから測った。実験009の教訓を使い、毎回新しいノードを作って初回のクックを測る。414,720点の皮で Linear 14.66ミリ秒、Dual Quaternion 15.21ミリ秒。倍率 1.04。点1つあたりの差は約 1.8ナノ秒。5通りの大きさを倍率は 0.98~1.12 に収まる。速さは理由にならない。

ねじれでも Linear は不利だった。骨の軸まわりに 179度ひねると、Linear は体積が 66.81% 減る。有名な「飴の包み紙」の形になり、真ん中がほぼ潰れる。Dual Quaternion は 0.24% しか減らない。

ただし Dual Quaternion にも癖があった。ねじれが根元まで広がる。170度を指定したとき、根元の帯（高さ 0~0.5）が Linear では 14.1度しかねじれないのに、Dual Quaternion では 58.9度ねじれる。そのうえ、高さ 1.10~1.50 の帯が 99.2度で止まっているのに、いちばん上の帯は 170度。途中に段差ができる。絵にもはっきり出る。

つまり「なぜ Linear が既定なのか」は、この2つの実験では決まらなかった。速さでもなく、ねじれの体積でもない。分かったのは、Dual Quaternion が別の癖を持っていることだけである。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwran gle	angle = 170.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotat(m, radians(ch("angle"))), set(0, 1, 0)); ...

ノード	タイプ	デフォルトから変更したパラメータ
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	method = 1

骨の軸まわりに 170度ひねる

皮は 1,440点の筒。骨は関節3つ。カメラは2枚とも同じ。



Houdini



Houdini

Linear。真ん中がほぼ潰れる。179度で体積は 66.81% 減る。

Dual Quaternion。太さは保たれるが、途中に段差ができる。

ひねり	Linear	減った分	Dual Quaternion	減った分
0°	0.636074	0.0000%	0.636074	0.0000%
45°	0.574068	9.7482%	0.636140	-0.0105%
90°	0.423900	33.3568%	0.635838	0.0371%
135°	0.273535	56.9964%	0.635251	0.1294%
170°	0.214342	66.3024%	0.634689	0.2177%
179°	0.211093	66.8131%	0.634541	0.2409%

速さ（毎回新しいノードを作って、初回のクックを測る）

結び付け（capture）までは先に計算を済ませてあるので、測っているのは皮を動かす処理だけ。1回目は準備の分が乗るので捨てている。

点の数	面の数	Linear	Dual Quaternion	倍率	1点あたりの差
2,880	2,858	4.47 ms	4.36 ms	0.98	-36.15 ns

点の数	面の数	Linear	Dual Quaternion	倍率	1点あたりの差
11,520	11,474	4.81 ms	5.41 ms	1.12	51.68 ns
46,080	45,986	5.75 ms	5.76 ms	1.00	0.20 ns
184,320	184,130	9.06 ms	9.78 ms	1.08	3.88 ns
414,720	414,434	14.66 ms	15.21 ms	1.04	1.32 ns

高さごとに、実際は何度ねじれたか（170度を指定）

点の x-z 平面での角度が、元の位置から何度動いたかの中央値。軸の近くは角度が不安定なので、外側だけを使っている。

元の高さ	Linear	Dual Quaternion
0.00～0.50	14.1°	58.9°
0.50～0.90	59.7°	80.6°
0.90～1.10	102.6°	87.9°
1.10～1.50	142.2°	99.2°
1.50～2.01	170.0°	170.0°

180度を越えるとうなるか

Dual Quaternion

は角度が半回転を越えると近道を通る、と言われる。体積で見ると限りそうはならなかった。

ひねり	Linear 減った分	Dual Quaternion 減った分
170°	66.3024%	0.2177%
179°	66.8131%	0.2409%
181°	66.8149%	0.5458%
200°	64.8223%	0.4665%
270°	33.4623%	0.1981%
350°	0.5167%	0.0107%

わかったこと

- ・速さは理由ではなかった。414,720点で Linear 14.66ミリ秒、Dual Quaternion 15.21ミリ秒。倍率 1.04。5通りの大きさが 0.98～1.12 に収まる。点1つあたりの差は約 1.8ナノ秒。
- ・ねじれでも Linear は不利だった。179度で体積が 66.81% 減る。真ん中がほぼ潰れて、有名な「飴の包み紙」の形になる。Dual Quaternion は 0.24%。
- ・ただし Dual Quaternion にも癖がある。ねじれが根元まで広がる。170度を指定したとき、根元の帯（高さ 0～0.5）は Linear で 14.1度しかねじれないのに、Dual Quaternion では 58.9度。動かしていない側まで巻き込む。
- ・そのうえ途中に段差ができる。高さ 1.10～1.50 の帯が 99.2度で止まっているのに、いちばん上の帯は 170度。その境目が絵でも見える。Linear は 14 → 60 → 103 → 142 → 170 と、なめらかにつながっている。

- ・ 180度を越えても、体積では破綻しなかった。Dual Quaternion は半回転を越えると近道を通る、と言われる。181度で 0.55%、270度で 0.20%、350度で 0.01%。少なくとも体積で見るかぎり、そうはならない。向きまでは調べていないので断定はしない。

- ・ Linear は 350度で元に戻る。減りが 0.52% まで下がる。一周に近づけば、ねじれていないのと同じだから。66.8% でいちばん潰れるのは 179度 = 半回転のところ。

- ・ 結局「なぜ Linear が既定なのか」は決まらなかった。速さでもなく、ねじれの体積でもない。分かったのは、Dual Quaternion が「ねじれを局所に留めない」という別の癖を持つことだけ。

- ・ 分からなかったことを、分からないまま残す。ここで「互換性のためだろう」と書くのは簡単だが、測っていない。この実験で言えるのは、速さとねじれの体積という2つの筋が消えた、ということ。

- ・ 測り方について。結び付け (capture) までは先に計算を済ませてから測っている。そうしないと、両方に共通の重い処理が時間に混ざる。比べたいものだけを測る形に作り替える。

次にやること

- ・ ねじれの向きまで調べて、180度越えの挙動を確かめる
- ・ 関節の回る範囲を決める (kinexfx::configurejointlimits)
- ・ APEX のグラフから骨を動かす